

— RESUMO DE PROJETO

PhotoApp

Aplicativo self-hosted de organização, edição e compartilhamento de fotos, com catalogação inteligente, edição não-destrutiva e dois recursos de IA rodando 100% localmente no servidor — busca semântica e aprimoramento de imagem.

Preparado para

Fernando José da Silva Melo

Data

22 de setembro de 2026

Repositório

github.com/fernandojsmelo/photoapp

Estado

18 commits · em produção local

— 01 · SUMÁRIO EXECUTIVO

O que é o PhotoApp

O PhotoApp nasceu de uma necessidade concreta: organizar, editar e (com segurança) compartilhar um acervo de fotos pessoais sem depender de nuvens de terceiros. O projeto foi construído do zero — do documento de requisitos ao deploy — em um único fluxo de trabalho iterativo, evoluindo em 14 fases distintas descritas na linha do tempo abaixo.

Hoje o sistema roda como um único container Docker (frontend + API + modelos de IA embutidos), suporta múltiplas contas administradas por um usuário administrador, e oferece dois recursos de inteligência artificial — busca por significado e aprimoramento de resolução — que rodam inteiramente no próprio servidor, sem qualquer chamada a serviços externos. Nenhuma foto ou metadado sai da máquina onde o PhotoApp está hospedado.

O diferencial mais recente é o modelo de compartilhamento: em vez do modelo tudo-ou-nada comum em apps desse tipo, o dono de uma foto escolhe exatamente quais imagens — dentro de um álbum ou avulsas — cada pessoa específica pode ver, e gerencia esses compartilhamentos em lote a qualquer momento.

Como o projeto foi construído

Cada fase corresponde a um ou mais commits no repositório, em ordem cronológica real de desenvolvimento.

1

Documento de Requisitos (PRD)

Definição de visão, personas, escopo MVP vs. futuro, requisitos funcionais e não-funcionais para um gerenciador de fotos avançado — catalogação, edição, álbuns/tags e IA para aprimoramento e busca.

PRD.md

planejamento

2

Primeira versão do frontend (v0 local)

React + Vite + TypeScript com persistência via IndexedDB — biblioteca de fotos, importação, visualizador, edição básica — testável no navegador sem nenhum backend.

React

IndexedDB

Vite

3

Fechamento da Fase 1

Associação de fotos a álbuns em massa (seleção múltipla), ferramenta de crop real no editor, sistema completo de tags (nuvem na barra lateral + autocomplete) e um placeholder local de "IA" por heurísticas de cor.

crop

tags

seleção em massa

4

Backend self-hosted

API Node.js + Express + TypeScript com armazenamento em SQLite (better-sqlite3) e arquivos originais/thumbnails em disco — substituindo por completo o IndexedDB do frontend.

Express

SQLite

API REST

5

Frontend migrado para a API

Toda a camada de dados do frontend (store Zustand) passa a falar com o backend real via HTTP, com autenticação por cookie — fim da dependência do navegador para persistência.

Zustand

fetch

6

Empacotamento em Docker

Frontend e backend passam a rodar juntos num único container, numa única porta, com dados persistidos em volume nomeado — deploy de um comando (`docker compose up`) em qualquer servidor doméstico ou VPS.

`Dockerfile``docker-compose`

7

Busca semântica real por IA

Modelo CLIP (Xenova/clip-vit-base-patch32) rodando localmente via `transformers.js` — compara o significado de uma descrição em texto com o embedding de cada foto, sem qualquer API externa. Modelo baixado e cacheado na build da imagem Docker.

`CLIP``transformers.js``100% local`

8

Multiusuário administrado

Autenticação com JWT em cookie `httpOnly` e senhas com `bcrypt`; o primeiro usuário criado vira administrador e passa a poder cadastrar novas contas pela tela "Usuários" — sem auto-registro público.

`JWT``bcrypt``admin`

9

Correções de experiência real de uso

Ajustes encontrados em teste manual: proxy do Vite para a API (evita depender de porta exposta do backend), preenchimento automático do navegador vazando credenciais do admin no formulário de novo usuário, botão de cancelar sem efeito, e validação de nome de usuário duplicado.

`proxy``UX``validação`

10

Filtros de cor também no servidor

Os ajustes de brilho/contraste/saturação/exposição/preset, antes só aplicados via CSS no navegador, passam a ser replicados no servidor (biblioteca `sharp`) na geração da miniatura — grade sempre fiel à edição salva.

`sharp``thumbnails`

11

HTTPS opcional via Caddy

Reverse proxy Caddy incluso no docker-compose (profile opt-in `https`) com emissão e renovação automática de certificado Let's Encrypt — pronto para expor o servidor na internet com um domínio (ex.: via DuckDNS).

Caddy

Let's Encrypt

12

Aprimoramento de foto com IA real

O placeholder heurístico de "auto-aprimorar" é substituído por um modelo real de super-resolução (Swin2SR, 2x), também rodando localmente — sem GPU, ~2,7s por foto.

Swin2SR

super-resolução

13

Álbuns compartilhados (somente leitura)

Uma conta pode compartilhar um álbum com outra conta do mesmo servidor. Quem recebe vê e navega as fotos, mas não edita, remove ou apaga nada — todas as rotas de escrita continuam restritas ao dono.

compartilhamento

somente leitura

14

Compartilhamento granular e gerenciamento em lote

Compartilhar um álbum deixa de liberar automaticamente todas as suas fotos — o dono escolhe exatamente quais ficam visíveis para cada pessoa. Passa a existir também compartilhamento de fotos avulsas, sem vínculo de álbum, e uma tela dedicada para revogar múltiplos compartilhamentos de uma vez, agrupados por pessoa.

granular

gerenciamento em lote

O que o sistema faz hoje

- **Catálogo e biblioteca**

Importação por arraste-e-solte ou seletor de arquivos, com deduplicação automática por hash do conteúdo (SHA-256). Extração de metadados EXIF (data de captura, modelo da câmera, geolocalização). Biblioteca com busca por nome/tag, favoritos e grade responsiva.

- **Edição não-destrutiva**

Brilho, contraste, saturação, exposição, rotação, recorte (crop) e cinco presets de cor (vívido, monocromático, quente, frio, esmaecido). O arquivo original nunca é alterado — os ajustes ficam salvos como metadados e são recalculados a cada visualização, inclusive na miniatura gerada pelo servidor.

- **Álbuns e tags**

Álbuns de lista plana, tags livres com autocomplete e nuvem de tags na barra lateral, seleção múltipla de fotos para ações em massa (adicionar a álbum, marcar com tag, favoritar, excluir).

- **Busca por IA (semântica)**

Descreva o que procura em texto livre ("praia ao pôr do sol", "aniversário com bolo") e o sistema rankeia as fotos por relevância de significado, comparando embeddings gerados pelo modelo CLIP — sem depender de tags ou nomes de arquivo.

- **Aprimoramento por IA**

Aumenta a resolução de uma foto em 2x usando um modelo de super-resolução real (Swin2SR), com nitidez preservada. Roda no servidor; a foto nunca sai dele.

- **Multiusuário e autenticação**

Cada pessoa tem sua própria conta, senha (hash bcrypt) e biblioteca isolada. O administrador cadastra novas contas pela tela "Usuários"; não existe cadastro público auto-serviço.

- **Compartilhamento granular**

- **Por álbum** — o dono escolhe quais fotos específicas do álbum ficam visíveis para cada pessoa (editável depois).
- **Avulso** — fotos soltas, sem vínculo de álbum, compartilhadas direto pelo visualizador ou por seleção múltipla.
- **Sempre somente leitura** — quem recebe nunca edita, remove ou apaga nada do dono.
- **Gerenciamento em lote** — painel dedicado que agrupa por pessoa e permite revogar vários compartilhamentos de uma vez.

- **Deploy e infraestrutura**

Um único container Docker com frontend, API e os dois modelos de IA já embutidos na imagem (build-time download) — sobe 100% offline. HTTPS automático opcional via Caddy + Let's Encrypt, ativado por profile do docker-compose.

Stack tecnológico

CAMADA	TECNOLOGIA	PAPEL
Frontend	React 19 + Vite + TypeScript	Interface e roteamento de telas
Estado	Zustand	Store global de fotos, álbuns e sessão
Backend	Node.js + Express 5 + TypeScript	API REST
Banco de dados	SQLite via better-sqlite3	Metadados, usuários, compartilhamentos
Imagens	sharp	Redimensionamento, crop, filtros de cor no servidor
IA local	@huggingface/transformers	Execução dos modelos CLIP e Swin2SR sem serviço externo
Autenticação	jsonwebtoken + bcryptjs	Sessão via cookie httpOnly + hash de senha
Upload	multer	Recebimento de arquivos (até 30 por vez, 30MB cada)
Metadados	exifr	Leitura de EXIF (data, câmera, GPS)
Deploy	Docker + docker-compose + Caddy	Empacotamento único e HTTPS automático

O que está pronto e o que fica para depois

- Em uso — testado ponta a ponta

- + **Pendente por decisão do usuário:** teste de HTTPS de ponta a ponta com domínio público real (DuckDNS) — a infraestrutura já está pronta no Caddy, falta apenas a configuração de rede/DNS do lado do usuário.
- + **Fora do escopo atual:** suporte a arquivos RAW, reconhecimento facial, álbuns aninhados e importação automática de outros serviços (Google Photos/Apple Photos) — todos assumidos como fase futura já no PRD original.
- + **Gerenciamento em lote** cobre hoje apenas compartilhamentos avulsos; compartilhamentos de álbum continuam geridos individualmente pelo modal de cada álbum.